Hospital 코드 리뷰

1. 프로젝트 개요

- 한 줄 설명: MEDIC-CRM(Next-HMS) — Next.js App Router + Supabase Realtime 기반 병원 행정/의료진 지원 시스템. 예약, 실시간 대기열, 환자 360 CRM, 실험실 검사 입력을 하나의 다크 테마 대시보드로 묶는 것이 목표.
- 목적/대상 사용자: 병원장(admin), 의료진(doctor), 원무과(staff), 임상병리사(lab_tech) 4개 역할의 내부 직원.

구분	기술
프론트	Next.js 16.2.9 (App Router, src/proxy.ts), React 19.2, Tailwind CSS v4, lucide-react, recharts 3
백엔드	별도 서버 없음. Supabase 클라이언트 직접 호출 (서버 컴포넌트 + 브라우저 클라이언트), Route Handler 2개
DB	Supabase PostgreSQL (patients, staff_profiles, appointments, lab_results 4 테이블, 스키마는 CLAUDE.md에만 존재)
인프라	로컬 개발 상태, Vercel 배포 전제. 마이그레이션 폴더/CI 없음
AI	없음

- 현재 완성도 추정: 약 45%
- 근거: 로그인/세션 유지, 대기열 실시간 조회·상태 변경, 캘린더 조회, 실험실 수치 입력은 실제 DB와 연동되어 동작한다. 반면 환자 등록, 예약 생성, 검사 의뢰, 진료 기록 작성 등 "생성(Create)" 기능은 버튼만 있고 핸들러가 없다(src/app/(dashboard)/patients/page.tsx:25-28, src/app/(dashboard)/lab/lab-client.tsx:113-115). 대시보드 KPI 변동률과 차트는 전부 하드코딩이다. PRD 3.1이 요구하는 RLS와 미들웨어 역할 분기는 미구현이다.
- 핵심 기능 목록
- 이메일 로그인 + 역할 선택 UI, 역할별 홈 리다이렉트
- 관리자 대시보드(환자 수, 금일 예약, 노쇼 건수 + 목업 차트)
- 환자 목록 / 환자 상세(기본정보, CRM 태그, 알레르기 경고, 예약 기반 타임라인)
- 월/주/일 예약 캘린더(의사 필터, 조회 전용)
- 실시간 대기열(Supabase Realtime, 진료 시작/완료/부재 처리, 의료진 가용 토글)
- 실험실 검사 수치 입력 + 정상범위 이탈 시 Medical Red 경고
- 권한 없는 페이지에 대한 "비밀번호 임시 접근" 게이트

2. 구조 분석

경로	역할
src/proxy.ts	Next 16 Proxy(구 middleware). 세션 쿠키 갱신, 비로그인 보호 경로 차단, /·/login 역할별 리다이렉트
src/lib/supabase/client.ts, server.ts	@supabase/ssr 브라우저/서버 클라이언트. Database 제네릭 적용
src/lib/permissions.ts	역할 -> 접근 가능 페이지 매트릭스(ROLE_ACCESS), canAccess, getPageKey
src/lib/lab-ranges.ts	CBC/LFT/소변검사 파라미터 정상 범위 상수와 isOutOfRange
src/app/(dashboard)/layout.tsx	서버에서 user/profile 조회 후 Sidebar 렌더. 비로그인 시 /login
src/components/layout/with-page-guard.tsx, page-guard.tsx	서버에서 role 조회 -> 클라이언트 PageGuard가 접근 허용/잠금 화면/비밀번호 모달 결정
src/app/api/auth/verify-admin/route.ts	OVERRIDE_PASSWORD 평문 비교 API
src/app/api/auth/callback/route.ts	OAuth code 교환(현재 OAuth 로그인 UI 없음)

경로	역할
src/app/(dashboard)/dashboard/page.tsx + components/dashboard/	KPI 카드, 매출/진료과 차트(목업), 최근 예약 테이블
src/app/(dashboard)/patients/page.tsx, [id]/page.tsx	환자 목록, 환자 360 상세
src/app/(dashboard)/calendar/	서버에서 의사 목록 -> 클라이언트가 전체 예약을 fetch하여 월/주/일 렌더
src/app/(dashboard)/queue/, components/queue/, hooks/use-realtime-queue.ts	대기열 페이지. 훅이 appointments 변경 이벤트마다 재조회
src/app/(dashboard)/lab/	검사 대기 목록 + 우측 수치 입력 패널
src/components/ui/ (10개)	shadcn/base-ui 컴포넌트. 페이지 코드에서 import 0건
src/types/supabase.ts, index.ts	수기 작성한 Database 타입과 도메인 타입
PRD.md, CLAUDE.md, Design_UI.md	기획서, DB 스키마(SQL), 디자인 토큰/프롬프트

데이터 흐름: 요청은 proxy.ts에서 세션만 확인한 뒤 (dashboard)/layout.tsx로 들어간다. 각 페이지는 서버 컴포넌트에서 anon key + 사용자 쿠키로 Supabase를 읽고, 그 결과를 WithPageGuard의 children으로 넘긴다. PageGuard(클라이언트)가 ROLE_ACCESS를 보고 children을 그대로 보여줄지 잠금 화면을 보여줄지 결정한다. 쓰기(상태 변경, 수치 저장, 가용 토글)는 전부 브라우저 클라이언트가 Supabase 테이블을 직접 update한다. 서버 액션은 없고, DB 측 RLS 정의도 저장소에 없다. 즉 권한 통제는 전부 UI 레이어에 있다.

3. 발견된 문제점

#	심각도	분류	위치	내용
1	높음	보안	src/components/layout/with-page-guard.tsx:11-27, src/components/layout/page-guard.tsx:22,36	역할 접근제어가 클라이언트 상태(unlocked)로만 결정됨. 서버 컴포넌트 children은 잠금 상태여도 RSC 페이로드에 포함되어 전송됨
2	높음	보안	src/app/api/auth/verify-admin/route.ts:6-9, .env.local (OVERRIDE_PASSWORD=1234)	공유 평문 비밀번호 1개로 모든 역할 제한 우회. 기본값 "1234", 시도 제한/세션 바인딩/감사 로그 없음
3	높음	보안	src/proxy.ts:43-48, 57, 69	Proxy는 로그인 여부만 검사하고 역할별 경로 차단 없음. 프로필이 없으면 role을 "admin"으로 기본 처리
4	높음	보안	CLAUDE.md 스키마 전체, src/app/(dashboard)/queue/queue-client.tsx:35-37,45-47, src/components/queue/live-queue-table.tsx:26-28, src/app/(dashboard)/lab/lab-client.tsx:95-100	RLS 정책이 저장소 어디에도 없고, 브라우저가 anon key로 appointments/staff_profiles/lab_results를 직접 update. 로그인한 어떤 역할이든 임의 레코드 변경 가능
5	높음	버그	src/app/(dashboard)/dashboard/page.tsx:41,53-54,59, src/app/(dashboard)/queue/page.tsx:16-21, src/hooks/use-realtime-queue.ts:14,23-24	toISOString().split("T")[0]은 UTC 날짜. 타임존 없는 ${today}T00:00:00과 결합되어 KST 00:00~09:00 데이터가 전날로 집계됨
6	중간	보안	src/app/(dashboard)/patients/[id]/page.tsx:36-53	유일하게 WithPageGuard 미적용. lab_tech도 URL 직접 입력으로 환자 상세 열람 가능
7	중간	보안	src/app/(auth)/login/page.tsx:66-76	역할 불일치 검사가 클라이언트 전용이며 프로필이 없으면 selectedRole을 그대로 신뢰
8	중간	버그	src/app/(dashboard)/queue/queue-client.tsx:23, src/components/queue/live-queue-table.tsx:19	같은 화면에서 useRealtimeQueue를 두 번 호출 -> Realtime 채널 2개, 동일 쿼리 2회, 카운트와 테이블이 서로 다른 스냅샷을 볼 수 있음
9	중간	버그	src/components/queue/live-queue-table.tsx:22-30, src/app/(dashboard)/queue/queue-client.tsx:40-48	PRD 4의 상태 전이 규칙(scheduled->checked_in->in_consultation->completed) 검증 없음. 어떤 상태에서든 임의 상태로 update 가능. canceled를 "부재/노쇼"로 표기(dashboard/page.tsx:19)해 취소와 노쇼가 섞임

#	심각도	분류	위치	내용
10	중간	버그	src/app/(dashboard)/calendar/calendar-client.tsx:69,325	스키마에 없는 visit_type 컬럼을 읽음(src/types/supabase.ts:105-112). 항상 null -> 진료 유형 범례(:16-20,207-219)는 죽은 기능
11	중간	품질	src/app/(dashboard)/dashboard/page.tsx:69-71,77,83,88, src/components/dashboard/revenue-chart.tsx:13-20, src/components/dashboard/department-chart.tsx:5-10	KPI 변동률(8.2, 3.1, -12.5, 5.7), 매출 "KRW 4.8M", 매출 추이·진료과 비율 차트가 전부 하드코딩. "오늘 총 환자" 카드가 실제로는 전체 환자 수
12	중간	품질	src/app/(dashboard)/patients/[id]/page.tsx:89,164-166	성별 "남" 하드코딩, 타임라인 유형을 idx % 4로 배정(실데이터와 무관). PRD 3.2가 요구한 lab_results 병합 없음
13	중간	구조	src/app/(dashboard)/patients/page.tsx:25-28,34-38, patients/[id]/page.tsx:70-75,149-156,201-204, lab/lab-client.tsx:113-115, calendar/calendar-client.tsx:85	신규 환자 등록, 검색, 예약 등록, 진료 시작, 기록 등록, 결과지 보기, 신규 검사 의뢰, 캘린더 슬롯 클릭 예약(PRD 3.4) 전부 onClick 없음
14	중간	품질	src/proxy.ts:52, src/app/(dashboard)/layout.tsx:16, with-page-guard.tsx:16, dashboard/page.tsx:61, patients/page.tsx:16, [id]/page.tsx:41,49, queue/page.tsx:11, queue-client.tsx:35,45, live-queue-table.tsx:26, lab/page.tsx:9, lab-client.tsx:95, calendar/page.tsx:8, calendar-client.tsx:116, use-realtime-queue.ts:17, login/page.tsx:61	createServerClient<Database>를 쓰면서도 모든 쿼리를 as any로 캐스팅(eslint-disable 20건). 타입 안전성 0, CLAUDE.md 3절 지침 위반
15	중간	버그	src/app/(dashboard)/lab/lab-client.tsx:66,92,98,101-102	저장 후 results 상태 미갱신(화면이 stale), 빈 입력을 parseFloat(v) 실패 시 0으로 대체해 저장, 상태를 항상 analyzing으로만 변경(승인 단계 미구현)
16	낮음	성능	src/app/(dashboard)/calendar/calendar-client.tsx:116-118	날짜 범위 없이 전체 appointments fetch. Realtime 미연동
17	낮음	성능	src/hooks/use-realtime-queue.ts:46-50	테이블 전체 변경 이벤트를 필터 없이 구독하고 매번 전체 재조회. doctor_id 필터와 payload 기반 부분 갱신 가능
18	낮음	구조	src/components/ui/ (10파일), package.json (zustand, shadcn, @base-ui/react)	shadcn UI 10개 컴포넌트는 페이지에서 import 0건. zustand 미사용, shadcn CLI가 runtime dependency에 포함
19	낮음	품질	src/proxy.ts:4-9,51-59,61-71, src/app/(auth)/login/page.tsx:16-28, src/components/layout/sidebar.tsx:27-32	ROLE_HOME/ROLE_LABEL/프로필 조회 로직이 3곳에 복제
20	낮음	보안	src/components/layout/sidebar.tsx:84-95	권한 없는 메뉴를 자물쇠 아이콘과 함께 클릭 가능하게 노출해 우회 경로를 안내
21	낮음	설정	README.md, .env.local, Hospital/.git	README가 create-next-app 기본값. 비밀값(OVERRIDE_PASSWORD)이 .env.local에 있고 상위 platform 저장소와 별도 nested git이 공존
22	낮음	버그	src/app/(dashboard)/calendar/calendar-client.tsx:26-33	주별 뷰가 월~금 5일만 생성. 주말 예약은 주별 뷰에서 보이지 않음

심각도 높음 상세

#1 클라이언트 전용 페이지 가드. WithPageGuard는 서버에서 role을 조회하지만 최종 판단은 클라이언트 PageGuard가 canAccess(role, page) || unlocked로 한다. children은 <DashboardContent /> 같은 서버

컴포넌트이고 React는 이를 먼저 서버에서 렌더해 RSC 페이로드에 담아 보낸다. 따라서 lab_tech가 /patients에 접근하면 화면에는 잠금이 뜨지만 네트워크 응답에는 전체 환자 목록이 이미 들어있다. 또 React DevTools로 unlocked를 true로 바꾸는 것만으로 우회된다. 가드는 서버(Proxy 또는 서버 컴포넌트에서 redirect)에서 끝나야 하고, children은 허용된 경우에만 렌더해야 한다.

#2 공유 오버라이드 비밀번호. verify-admin 라우트는 요청 바디의 평문 비밀번호를 환경변수(기본 "1234")와 ===로 비교하고 {ok:true}만 반환한다. 성공해도 서버 세션에는 아무 변화가 없으므로 실질적 효과는 클라이언트 상태 토글이다. 의료 데이터 시스템에서 "누구나 아는 숫자 4자리"로 모든 역할 제한을 넘는 설계는 PRD 3.1의 "민감 의료기록 API 탈취 방어"와 정면으로 충돌한다. 기능을 제거하거나, 최소한 admin 계정 재인증(Supabase signInWithPassword로 본인 비밀번호 확인) + 서버 측 단기 권한 상승 토큰으로 바꿔야 한다.

#3 Proxy의 역할 미검사와 admin 기본값. PROTECTED_PREFIXES는 로그인 여부만 본다. 역할별 경로 차단은 전무하며, /login·/ 리다이렉트 시 프로필 조회가 실패하면 role = "admin"으로 처리해 가장 높은 권한 홈으로 보낸다. with-page-guard.tsx:21은 같은 상황에서 "staff"를 기본값으로 쓰고 있어 파일마다 기본 권한이 다르다. 프로필이 없는 계정은 로그인 자체를 거부해야 한다.

#4 RLS 부재 + 브라우저 직접 쓰기. 저장소에 SQL 마이그레이션이 없고 CLAUDE.md의 스키마에도 ENABLE ROW LEVEL SECURITY가 없다. 클라이언트는 anon key와 사용자 JWT로 appointments, staff_profiles, lab_results를 직접 update하므로 RLS가 없다면 로그인한 staff가 다른 의사의 is_available을 바꾸거나 lab 결과를 덮어쓸 수 있고, 있다 해도 정책이 코드로 관리되지 않아 검증 불가다. 쓰기는 서버 액션으로 모으고, 테이블별 RLS를 supabase/migrations에 커밋해야 한다.

#5 UTC 날짜 집계. 서버(Vercel)와 브라우저가 모두 UTC 기준 날짜 문자열을 만들고 타임존 없는 ISO 문자열로 비교한다. 한국 시간 오전 9시 이전에는 "오늘"이 전날로 계산되고, 00:00~09:00에 등록된 예약은 대기열에서 사라진다. 실시간 대기열이 핵심 가치인 제품에서 운영 시작 시간대에 데이터가 비는 치명적 UX 버그다. Asia/Seoul 기준 시작/끝 시각을 UTC ISO로 변환해 gte/lt로 비교해야 한다.

4. 효율성 개선 제안

- 구조/아키텍처
- 쓰기 경로를 src/app/actions/ 하위 .ts 파일 서버 액션으로 통일하고, 액션 내부에서 (a) staff_profiles.role 검사 (b) 상태 전이 테이블 검증 (c) DB 호출 순서로 고정한다. 클라이언트에는 createClient() 읽기 + Realtime 구독만 남긴다.
- 역할 라우팅을 한 곳으로 모은다: lib/permissions.ts에 ROLE_HOME, ROLE_LABEL, getPageKey를 두고 proxy.ts가 canAccess(role, getPageKey(pathname)) 실패 시 바로 redirect. WithPageGuard/PageGuard는 삭제.
- 환자 360 타임라인은 appointments와 lab_results를 서버에서 병합한 TimelineEntry[]로 만들어 넘긴다(이미 types/index.ts:24-32에 타입이 있음).
- 성능
- 캘린더는 현재 뷰의 날짜 범위만 조회하고 appointments Realtime을 구독해 즉시 반영.
- useRealtimeQueue를 QueuePageClient에서 한 번만 호출하고 LiveQueueTable에는 props로 내려준다. 구독 시 filter: doctor_id=eq.<id>를 쓰고 payload로 부분 갱신.
- 대시보드 KPI는 count: 'exact', head: true 4개 쿼리 대신 PostgreSQL 뷰 또는 RPC 1회로 집계.
- 개발 생산성
- supabase gen types typescript로 types/supabase.ts를 생성하고 as any 20건 제거. join 결과 타입은 QueryData<typeof query> 헬퍼 사용.
- 사용하지 않는 components/ui/, zustand, shadcn, @base-ui/react를 제거하거나 실제로 사용(현재 raw <button>/<input> 반복이 많아 shadcn을 쓰면 중복이 크게 줄어듦).
- 날짜 유틸(lib/date.ts: todayRangeKST(), formatHHMM)을 만들어 4곳의 날짜 코드를 통합.
- 최소 테스트: permissions.ts, lab-ranges.ts, 상태 전이 함수에 Vitest 단위 테스트.
- 운영/배포

- supabase/migrations/001_schema.sql(테이블 + RLS + staff_profiles 자동 생성 트리거)과 seed를 커밋. CLAUDE.md의 SQL은 이 파일을 참조하도록 변경.
- .env.example 추가, OVERRIDE_PASSWORD 제거. README를 프로젝트 전용 내용으로 교체.
- 상위 platform 저장소와 nested .git 중 하나로 정리.

5. 수정 작업 목록 및 권장 모델

#	작업	우선순위	예상 규모	권장 모델	선정 이유
1	supabase/migrations/001_schema.sql 작성: 4개 테이블 + 테이블별 RLS(역할은 staff_profiles 조회 함수 SECURITY DEFINER로 판정) + 가입 시 staff_profiles 생성 트리거	P0	M	Fable 5.1	보안·데이터 모델 핵심. 재귀 정책 등 실수 비용이 큼
2	proxy.ts에 역할 기반 경로 차단 구현, 프로필 없음 -> 로그아웃 처리, WithPageGuard/PageGuard/verify-admin 제거, patients/[id]도 동일 보호	P0	M	Fable 5.1	인증/권한 로직 재설계
3	쓰기 로직을 서버 액션으로 이전(actions/appointments.ts, actions/lab.ts, actions/staff.ts) + 상태 전이 검증 함수와 단위 테스트	P0	L	Fable 5.1	다중 파일에 걸친 보안 리팩토링
4	KST 날짜 범위 유틸 작성 후 dashboard/queue/hook 4곳 교체	P0	S	Opus 5.5	원인이 명확한 중간 난이도 버그
5	supabase gen types로 타입 재생성, as any 20건 제거, join 타입 정리	P1	M	Opus 5.5	2~5개 파일 이상 타입 정리
6	신규 환자 등록 모달 + 환자 검색(서버 액션, patients insert, 이름/전화 ilike)	P1	M	Opus 5.5	명확히 정의된 CRUD 기능
7	캘린더: 날짜 범위 조회, 빈 슬롯 클릭 -> 예약 생성 모달, Realtime 구독, 주말 포함 옵션	P1	M	Opus 5.5	API 연동 + 상태관리 정리
8	대시보드 실데이터화: 월별 매출/예약 집계 RPC 또는 뷰, 진료과 비율, 전일 대비 변동률 계산. 매출 컬럼 없으면 PRD 수납 기능과 함께 스키마 확장	P1	M	Opus 5.5	집계 쿼리 설계 + 차트 연동
9	환자 360 타임라인에 lab_results 병합, 성별 컬럼 추가 또는 표기 제거, 진료 기록 등록 액션	P1	M	Opus 5.5	데이터 병합 + 폼 구현
10	실험실: 저장 후 목록 갱신(router.refresh 또는 Realtime), 승인(approved) 단계, 신규 검사 의뢰 모달, 빈 값 저장 방지	P1	M	Opus 5.5	중간 규모 기능 보완
11	useRealtimeQueue 단일 호출로 정리, doctor_id 필터 구독, payload 부분 갱신	P2	S	Opus 5.5	상태관리/성능 정리
12	visit_type 컬럼 추가(마이그레이션 + 타입) 또는 캘린더 범례 제거	P2	S	Sonnet 5.5	단순 정리
13	미사용 components/ui/, zustand, shadcn 정리 또는 shadcn으로 raw 버튼/인풋 치환	P2	M	Sonnet 5.5	반복적 컴포넌트 작업
14	ROLE_HOME/ROLE_LABEL 중복 제거, README/.env.example 작성, nested git 정리	P2	S	Sonnet 5.5	문서화·설정 정리

6. 종합 평가

- 강점

1. Supabase Realtime을 활용한 대기열 동기화가 실제로 동작하고, 훅/테이블/카드로 역할이 분리되어 있다.
2. lib/permissions.ts, lib/lab-ranges.ts처럼 도메인 규칙을 상수화한 점과 Design_UI.md의 토큰(#0F172A/#E11D48)이 코드 전반에 일관되게 적용되어 UI 완성도가 높다.

3. 서버 컴포넌트에서 데이터를 읽고 클라이언트 컴포넌트에 props로 넘기는 기본 구조가 Next.js App Router 관례에 맞다.

- 약점

1. 권한 모델이 전부 클라이언트에 있고(가드, 비밀번호 우회, 직접 update) RLS가 없어 의료 데이터 시스템으로서 보안 요건을 충족하지 못한다.
2. "생성" 기능 대부분이 UI 껍데기이고 대시보드 지표가 목업이어서 PRD 대비 실제 가치 전달 비율이 낮다.
3. 타입 시스템을 우회(`as any` 20건)하고 UTC 날짜 버그가 핵심 화면 4곳에 걸쳐 있어 데이터 정합성 신뢰가 낮다.

- 모델별 비율: Fable 5.1 21% (3건) / Opus 5.5 57% (8건) / Sonnet 5.5 22% (3건)
- 권장 진행 순서

1. 1단계(보안 토대): 작업 1 -> 2 -> 3 -> 4. RLS와 서버 액션, Proxy 역할 차단, 날짜 버그를 먼저 닫아 "안전하게 데이터를 쓸 수 있는 상태"를 만든다.
2. 2단계(기능 완성): 작업 5 -> 6 -> 7 -> 8 -> 9 -> 10. 타입을 먼저 정리한 뒤 환자/예약/검사 생성 기능과 실데이터 대시보드를 채운다.
3. 3단계(정리): 작업 11 -> 12 -> 13 -> 14. 성능, 미사용 코드, 문서.

- 주 담당 모델: **Fable 5.1**. 남은 작업의 수는 Opus 비중이 크지만, 이 프로젝트의 가치는 "의료 데이터를 역할별로 안전하게 다루는가"에 달려 있고 그 부분(RLS 설계, Proxy 재설계, 쓰기 경로 서버 이전, 상태 전이 규칙)이 서로 얽혀 있어 한 모델이 전체 권한 모델을 일관되게 설계해야 한다. 보안 토대가 잡힌 뒤의 CRUD/차트 작업은 Opus 5.5에 위임하고, 스타일/정리 작업은 Sonnet 5.5로 내리는 것이 비용 효율적이다.